Cloud & DevOps Project Ideas — Beginner → Intermediate → Advanced

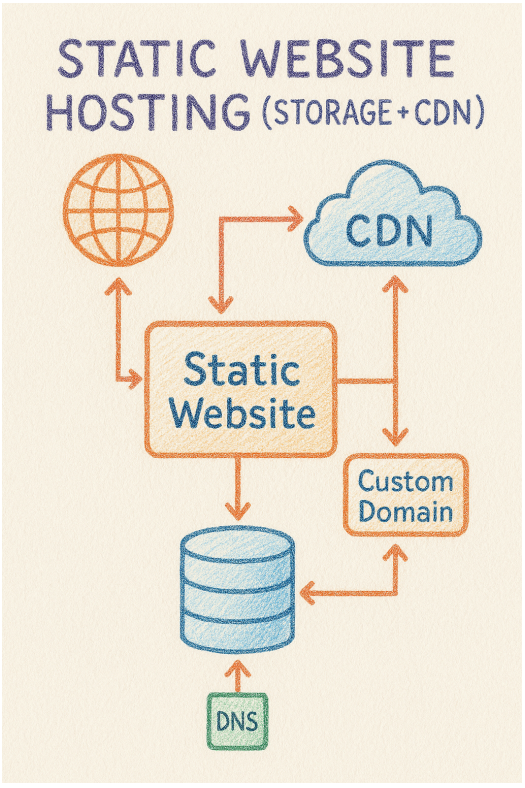
Each project includes: Level, Layman explanation, What it is about, Steps for AWS/GCP/Azure and What it depicts to an interviewer. Use GPT to create your project.

1) Static Website Hosting (Storage + CDN)

Level: Beginner

Layman Explanation: Put your resume/portfolio online so anyone can open it from anywhere — like uploading files but publicly accessible and fast.

What it is about: Host static HTML/CSS files on cloud object storage and use CDN for global delivery.



Steps (AWS / GCP / Azure):

AWS:

• Create an S3 bucket and enable Static Website Hosting.

• Set bucket policy or use CloudFront for secure access.

• Upload files using AWS CLI.

• Create CloudFront distribution and point to S3 origin.

• Configure Route53 record for custom domain.

GCP:

• Create a Cloud Storage bucket and set website configuration.

• Upload files using gsutil.

• Enable Cloud CDN and attach to backend bucket via Load Balancer.

• Configure Cloud DNS for custom domain.

Azure:

• Create a Storage Account and enable Static Website feature.

• Upload files to the $web container with Azure CLI.

• Create Azure CDN endpoint and point to storage account.

• Configure Azure DNS or custom domain mapping.

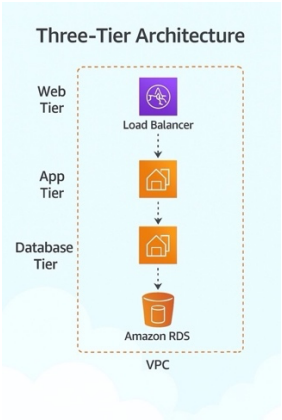
What this depicts to an interviewer: Shows knowledge of cloud storage, CDN, and basic DNS/domain setup.

2. Three-Tier Architecture (VPC/Network + Web + App + DB)

Level: Beginner

Layman Explanation: Like a shop with a storefront, a storeroom, and a locked safe — separate layers for users, business logic, and data.

What it is about: Provision VPC/Network, public web servers, private app servers, and managed database. Use load balancer in front.



Steps (AWS / GCP / Azure):

AWS:
Create a VPC: Define a Virtual Private Cloud (VPC) with public and private subnets across multiple Availability Zones.

Launch Servers: Deploy EC2 instances for the web tier into the public subnets and EC2 instances for the app tier into the private subnets.

Provision Database: Create a managed Relational Database Service (RDS) instance in the private subnets.

Set up Load Balancer: Configure an Application Load Balancer (ALB) to distribute external traffic to the web tier instances.

Configure Security Groups: Implement firewall rules to control traffic: allow traffic from the ALB to the web tier, from the web tier to the app tier, and from the app tier to the RDS database.

GCP:
Create a VPC Network: Set up a network and subnets.

Launch VMs: Deploy Compute Engine instances for the web tier and app tier.

Provision Database: Create a Cloud SQL instance and configure it with a private IP address for internal access only.

Set up Load Balancer: Configure an HTTP(S) Load Balancer with a backend service pointing to an instance group containing your web tier VMs.

Configure Firewall Rules: Use network tags and firewall rules to restrict traffic: allow traffic from the load balancer to the web tier, from the web tier to the app tier, and from the app tier to the Cloud SQL private IP.

Azure:

Create a VNet: Define a Virtual Network (VNet) with separate subnets for each tier.

Launch VMs: Deploy Virtual Machines for the web tier into the public subnet and for the app tier into a private subnet.

Provision Database: Create a managed database (like Azure SQL or Azure Database for PostgreSQL) and secure it within the VNet using a Private Endpoint.

Set up Load Balancer: Configure an Azure Load Balancer to distribute internet traffic to the web tier VMs.

Configure Network Security Groups (NSGs): Apply NSGs to each subnet to enforce traffic rules, ensuring communication only happens between the intended tiers (e.g., Web -> App, App -> DB).

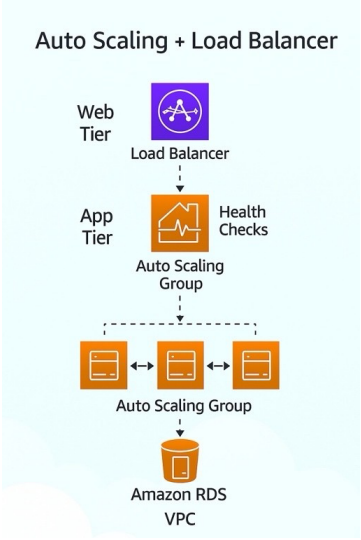
What this depicts to an interviewer: Shows understanding of cloud networking, segregation of concerns, and managed databases.

3)Auto Scaling + Load Balancer

Level: Beginner

Layman Explanation: Automatically bring more servers when lots of users come, and remove them when traffic drops.

What it is about: Set up managed auto-scaling with health checks behind a load balancer and demonstrate scaling.



Steps (AWS / GCP / Azure):

AWS:

Create a Launch Template: Define the blueprint for your EC2 instances (AMI, instance type, security group).

Create an Application Load Balancer (ALB): Set up a target group and a health check to monitor instance health.

Create an Auto Scaling Group (ASG): Link it to your launch template and the ALB's target group.

Configure Scaling Policies: Set rules for scaling, such as "add an instance if average CPU exceeds 70%" and "remove an instance if CPU drops below 30%."

GCP:

Create an Instance Template: Define the configuration for your VM instances (machine type, image, startup script).

Create a Managed Instance Group (MIG): Base it on the instance template.

Configure Autoscaling: Set the autoscaling policy for the MIG, based on metrics like CPU utilization or requests per second.

Set up an HTTP(S) Load Balancer: Create a backend service that uses the MIG as its backend and attach a health check

.**Azure:**

Create a Virtual Machine Scale Set (VMSS): Specify the OS image, instance size, and initial instance count.

Integrate a Load Balancer: During setup, create an Azure Load Balancer with a backend pool, health probe, and a rule to distribute traffic.

Configure Scaling Rules: In the VMSS "Scaling" settings, define rules for scaling out (adding instances) and scaling in (removing instances) based on metrics like CPU percentage.

Set Instance Limits: Define the minimum and maximum number of instances for the scale set.

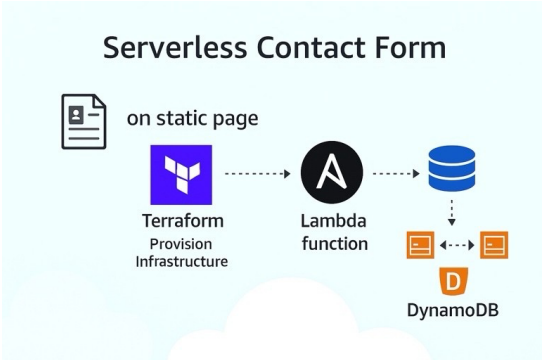
What this depicts to an interviewer: Shows skills in building resilient, elastic infrastructure.

4)Serverless Contact Form (API -> Function -> DB)

Level: Beginner

Layman Explanation: A contact form that sits on a static page; when someone submits, a tiny function runs and stores the entry.

What it is about: Static form triggers a cloud function via API to write to a managed NoSQL DB.



Steps (AWS / GCP / Azure):

AWS:

Create a DynamoDB Table: Set up a simple NoSQL table to store form entries.

Create a Lambda Function: Write code (e.g., in Python/Node.js) to process the incoming data and save it to DynamoDB.

Create an API Gateway Endpoint: Configure an HTTP endpoint that triggers your Lambda function on a POST request.

Update Frontend: Point your HTML form's action to the API Gateway URL.

GCP:

Create a Firestore Database: Set up a NoSQL database and a collection for your entries.

Create a Cloud Function: Write code that is activated by an HTTP request, parses the form data, and writes it to Firestore.

Deploy Function: Deploy the function to get a public trigger URL.

Update Frontend: Point your HTML form's action to the Cloud Function's URL.

Azure:

Create a Cosmos DB Account: Set up a database and a container for your data.

Create an Azure Function: Use an "HTTP Trigger" template. This gives you a public URL.

Add an Output Binding: Connect the function to your Cosmos DB container. The function code just needs to return the data to be saved.

Update Frontend: Point your HTML form's action to the Azure Function's URL.

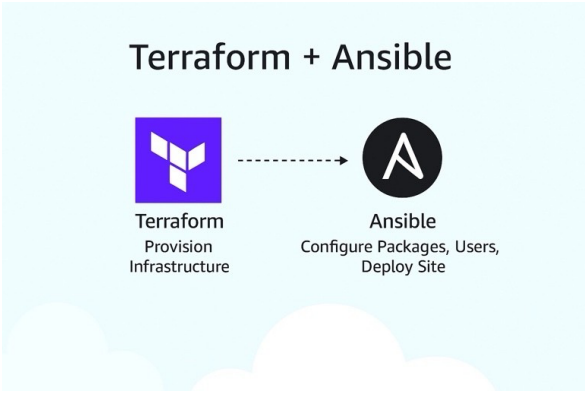
What this depicts to an interviewer: Shows ability to build event-driven, pay-per-use systems.

5)Terraform + Ansible (Infra provisioning + Configuration)

Level: Intermediate

Layman Explanation: Terraform lays the bricks; Ansible brings in furniture.

What it is about: Use Terraform to provision infra and Ansible to configure packages, users, and deploy site.



Steps (AWS / GCP / Azure):

AWS:

Terraform: Write code (.tf files) to define your AWS resources: a VPC, subnets, security groups (allowing SSH/HTTP), and one or more EC2 instances.

Provision: Run terraform apply. Terraform calls AWS APIs to create the resources. It is configured to output the public IP addresses of the created EC2 instances.

Ansible Inventory: Automatically or manually update an Ansible inventory file with the IP addresses from Terraform's output.

Configure: Run an ansible-playbook command. Ansible connects to the EC2 instances via SSH, installs packages (like Nginx or Apache), and copies your website files to the server.

GCP:

Terraform: Write code to define GCP resources: a VPC network, firewall rules (allowing SSH/HTTP), and one or more Google Compute Engine (GCE) VM instances.

Provision: Execute terraform apply. Terraform provisions the defined resources in your GCP project and outputs the public IPs of the GCE instances.

Ansible Inventory: Populate an Ansible inventory with the IP addresses of the new VMs.

Configure: Execute an Ansible playbook that SSHes into the GCE instances to install web servers, configure users, and deploy your site.

Azure:

Terraform: Write code to define Azure resources: a Resource Group, Virtual Network (VNet), Network Security Group (NSG) with rules for SSH/HTTP, and one or more Azure Virtual Machines (VMs).

Provision: Run terraform apply. Terraform creates the infrastructure in your Azure subscription and outputs the public IP addresses of the VMs.

Ansible Inventory: Use the output IPs from Terraform to create your Ansible inventory list.

Configure: Run an Ansible playbook. Ansible connects to the Azure VMs to perform configuration tasks like installing software and deploying your application code.

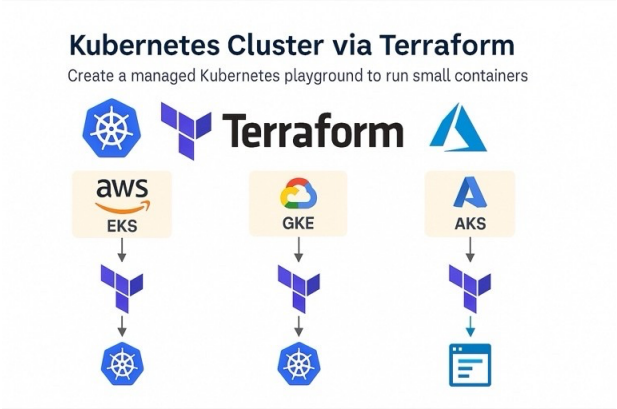
What this depicts to an interviewer: Shows practical Infra-as-Code plus configuration management workflow.

6) Kubernetes Cluster via Terraform (EKS / GKE / AKS)

Level: Intermediate

Layman Explanation: Create a managed Kubernetes playground to run small containers.

What it is about: Use Terraform to provision a managed cluster; deploy manifests and expose app.



Steps (AWS / GCP / Azure):

AWS:

Terraform Code: Write HCL code to define an Amazon EKS (Elastic Kubernetes Service) cluster, its associated VPC/subnets, IAM roles, and EC2-based node groups.

Provision Cluster: Run terraform apply. Terraform will create the cluster, and you can configure it to output a kubeconfig file for kubectl.

Kubernetes Manifests: Create standard Kubernetes files, including a deployment.yaml for your application containers and a service.yaml of type LoadBalancer.

Deploy Application: Use kubectl apply -f . to deploy your application. EKS will automatically provision an AWS Application Load Balancer to expose your service.

GCP:
Terraform Code: Write Terraform code to define a Google Kubernetes Engine (GKE) cluster and its node pools.

Networking is often simpler and can be managed by GKE.

Provision Cluster: Run terraform apply. Terraform creates the cluster, and you can use gcloud to get the credentials for kubectl.

Kubernetes Manifests: Create a deployment.yaml to define your pods and a service.yaml with type: LoadBalancer.

Deploy Application: Run kubectl apply -f .. GKE will provision a Google Cloud Load Balancer to provide an external IP for your service.

Azure:
Terraform Code: Write Terraform code to define an Azure Kubernetes Service (AKS) cluster, its node pools, and the

necessary service principal or managed identity for permissions.

Provision Cluster: Run terraform apply. Terraform builds the cluster, and you can use the Azure CLI to download the kubeconfig credentials.

Kubernetes Manifests: Create your application's deployment.yaml and expose it using a service.yaml of type LoadBalancer.

Deploy Application: Run kubectl apply -f . to deploy the manifests. AKS will automatically configure an Azure Load Balancer to expose your application to the internet.

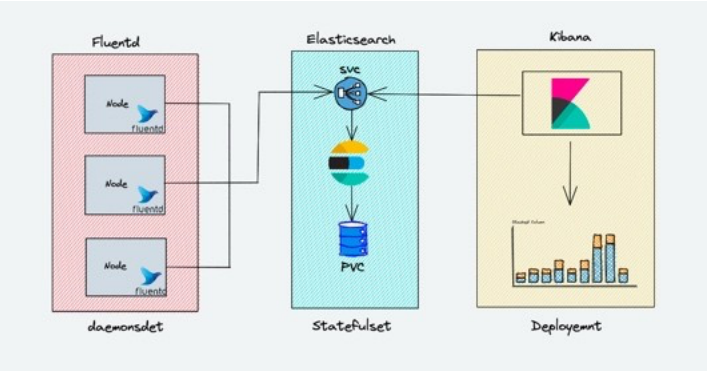
What this depicts to an interviewer: Shows ability to provision and run workloads on Kubernetes across cloud providers.

7)Observability: Prometheus + Grafana + EFK

Level: Intermediate

Layman Explanation: Dashboards and log collection so you can see app health and issues.

What it is about: Deploy monitoring and logging stack on Kubernetes using Helm.



Steps (AWS / GCP / Azure):

AWS/GCP/Azure:

• Helm install Prometheus, Grafana, EFK stack

Add Helm Repositories: Use the helm repo add command to tell Helm where to find the official charts (packages) for the monitoring and logging tools.

Install Monitoring Stack: Run helm install prometheus-community/kube-prometheus-stack. This single command deploys Prometheus for collecting metrics and Grafana for visualizing them in dashboards.

Install Logging Stack: Run helm install for each component of the EFK stack. First, install Elasticsearch to store logs, then Fluentd to collect them from across the cluster, and finally Kibana to provide a search interface.

Access Dashboards: Use kubectl port-forward to create a secure tunnel from your local machine to the Grafana and Kibana services running in the cluster, allowing you to view metrics and search logs in your web browser.

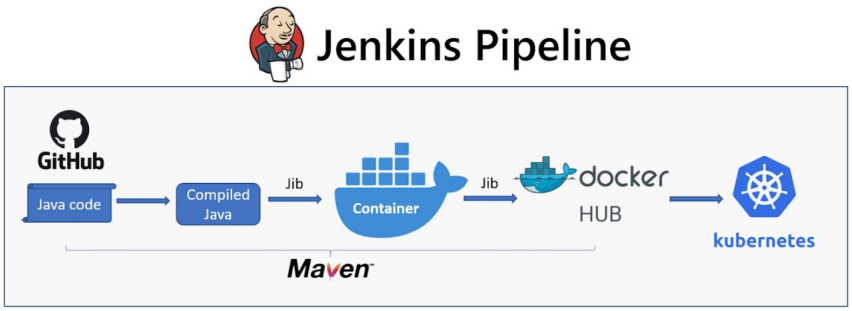
What this depicts to an interviewer: Shows monitoring, logging and alerting skills.

8)Jenkins Pipeline: Terraform + Ansible

Level: Advanced

Layman Explanation: Push code — Jenkins runs Terraform then Ansible automatically.

What it is about: Create a Jenkins pipeline to provision infra (Terraform) and configure it (Ansible).



Steps (AWS / GCP / Azure):

AWS/GCP/Azure:

Provision Jenkins, pipeline with Terraform + Ansible

The process is cloud-agnostic; the Jenkins pipeline structure remains the same, while the Terraform/Ansible code contains the cloud-specific details.

Provision Jenkins Server: Set up a Jenkins controller on a virtual machine in your cloud of choice. Install necessary plugins: Git, Terraform, Ansible, and Credentials Binding.

Configure Credentials: In Jenkins's credential manager, securely store the secrets needed for the pipeline:

Cloud Provider Credentials: An API key/secret (AWS), service account key (GCP), or service principal (Azure) for Terraform to authenticate.

SSH Key: An SSH private key for Ansible to securely connect to the newly created virtual machines.

Create a Jenkinsfile: Define the pipeline as code in a file named Jenkinsfile in your Git repository. The file will have multiple stages:

Stage 1 - Terraform Plan: Runs terraform plan to show a preview of changes, providing a manual check before proceeding.

Stage 2 - Terraform Apply: Runs terraform apply to provision the infrastructure. This stage is configured to output the IP addresses of the created servers into a temporary inventory file for Ansible.

Stage 3 - Ansible Configure: Runs the ansible-playbook command, pointing it to the inventory file created in the previous stage and using the stored SSH key to configure the servers.

Create and Trigger the Jenkins Job: In the Jenkins UI, create a new "Pipeline" job and point it to your Git repository. When a developer pushes a change to the repository, a webhook automatically triggers the Jenkins job, which reads the Jenkinsfile and executes the entire provisioning and configuration workflow without any manual intervention.

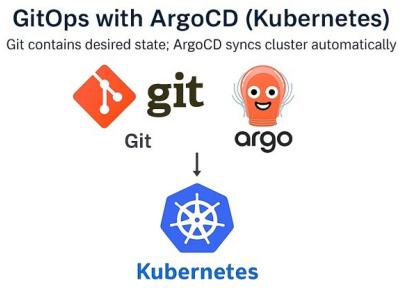
What this depicts to an interviewer: Shows CI/CD applied to infrastructure lifecycle.

9)GitOps with ArgoCD (Kubernetes)

Level: Advanced

Layman Explanation: Git contains desired state; ArgoCD syncs cluster automatically.

What it is about: Implement GitOps with ArgoCD: manifests in Git auto-sync to cluster.



Steps (AWS / GCP / Azure):

AWS/GCP/Azure:

• Install ArgoCD, connect repo, auto-sync

The process is identical for any standard Kubernetes cluster (like AWS EKS, GCP GKE, or Azure AKS).

Install ArgoCD on the Cluster: Apply the official ArgoCD installation manifest to your Kubernetes cluster. This creates a new argocd namespace and deploys the ArgoCD controller, API server, and other necessary components.

Connect the Git Repository: Configure an "ArgoCD Application" (either through the UI or a YAML manifest). This tells ArgoCD three key things:

The URL of your Git repository.

The path within the repository where your Kubernetes manifests are located.

The destination cluster and namespace where the application should be deployed.

Deploy the Application from Git: Add your application's Kubernetes manifests (e.g., deployment.yaml, service.yaml) to the specified path in your Git repository and push the changes.

Enable Auto-Sync: In the ArgoCD Application settings, enable the "automated sync policy." ArgoCD will now detect the new commit, see that the cluster's state is "OutOfSync" with the Git repository, and automatically apply the manifests to deploy your application. Any future commits pushed to that repository path will be automatically synchronized with the cluster.

What this depicts to an interviewer: Shows modern GitOps deployment practices.

10)Blue-Green / Canary Deployments on Kubernetes

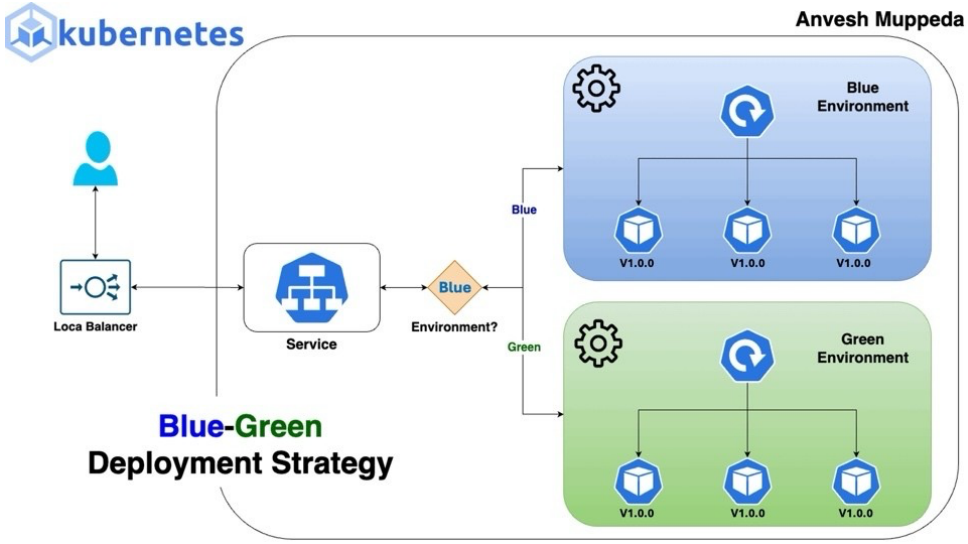
Level: Advanced

Layman Explanation: Run old and new versions side-by-side; shift traffic gradually.

What it is about: Implement blue/green or canary deployments with service selectors, Istio or Argo Rollouts.

Blue-Green Deployment: You run two identical, parallel environments: "Blue" (the current stable version) and "Green" (the new version). All live traffic goes to Blue. Once the Green environment is tested and ready, you switch the router to send 100% of traffic to Green. Blue is kept on standby for a quick rollback if needed.

Canary Deployment: You release the new version ("Canary") to a small subset of users first. You deploy the new version alongside the stable one and use a router to send a small percentage of traffic (e.g., 5%) to the Canary. You monitor its performance and error rates. If it's healthy, you gradually increase its traffic share until it handles 100%.



Steps (AWS / GCP / Azure):

AWS/GCP/Azure:

• Deploy v1+v2, switch traffic gradually

Manual Method (Using Kubernetes Service Selectors)

This is the most basic approach, suitable for simple Blue-Green deployments.

Deploy Two Versions: Create two separate Kubernetes Deployment resources, one for your stable version (v1) and one for the new version (v2). Use different labels for each, for example, version: blue and version: green.

Point Service to Blue: Create a Kubernetes Service that selects the pods for the stable "Blue" version by using its label selector (e.g., selector: { version: blue }). All traffic now goes to v1.

Test Green Internally: Access your "Green" deployment internally (e.g., via port-forwarding or a separate internal service) to ensure it works correctly.

Switch Traffic: To perform the Blue-Green switch, update the Service's label selector in a single command to version: green. Kubernetes will atomically switch all traffic from the Blue deployment to the Green one.

2. Service Mesh (Using Istio)

A service mesh provides fine-grained traffic control, making it perfect for automated canary deployments.

Deploy Both Versions: Deploy both v1 and v2 Deployments with distinct version labels (e.g., version: v1, version: v2). Both are active at the same time.

Define Traffic Rules: Create an Istio VirtualService resource. In this resource, define routing rules that specify what percentage of traffic goes to each version. For example, send 90% of traffic to v1 and 10% to v2.

Gradually Shift Traffic: To increase the canary percentage, simply update the weights in the VirtualService manifest and re-apply it (e.g., change from 90/10 to 50/50). Istio's proxies will adjust the traffic flow instantly without any changes to the underlying pods.

Complete the Rollout: Once the canary is deemed stable, adjust the VirtualService to send 100% of traffic to v2. You can then scale down the v1 deployment.

3. Progressive Delivery Controller (Using Argo Rollouts)

This is a purpose-built, automated solution that extends Kubernetes functionality.

Install Argo Rollouts: Apply the Argo Rollouts controller manifest to your cluster. This adds a new Rollout resource type to Kubernetes.

Create a Rollout Resource: Instead of a Deployment, define your application using a Rollout resource. In the spec, define a strategy (e.g., canary) with specific steps, such as "send 10% of traffic, then pause for 5 minutes for analysis, then increase to 50%".

Trigger the Rollout: To start a deployment, simply update the container image tag in your Rollout manifest and apply it (kubectl apply).

Automated Promotion or Rollback: Argo Rollouts automatically executes the steps you defined. It can be configured to query a metrics provider (like Prometheus) during the pause steps. If error rates are high, it will automatically roll back the deployment. If metrics are healthy, it will proceed to the next step until the rollout is complete.

What this depicts to an interviewer: Shows ability to perform zero-downtime releases and safe rollouts.